DÉPARTEMENT DE MATHÉMATIQUES-INFORMATIQUE

Licence 3 — Programmation Orientée Objet & Génie Logiciel

RAPPORT DE TRAVAUX PRATIQUES (TP)

UNITÉ D'ENSEIGNEMENT : DÉVELOPPEMENT D'APPLICATIONS EN C++

Analyse, conception UML et implémentation d'une application de gestion des vols et des réservations pour une agence de voyage à Cotonou

Type de document	**Compte-rendu de Travaux Pratiques (TP)**
Étudiant	**AKAKPO Yéma**
Niveau d'études	Licence 3
Filière	Mathématiques-Informatique
Sujet du TP	Gestion des vols et des réservations (C++)
Date de remise	13 novembre 2026

1. Sommaire

Section	Intitulé	Contenu principal
1	Sommaire & Résumé exécutif	Plan du rapport de TP et synthèse
2	Introduction & Présentation	Contexte, périmètre et acteurs du système
3	Analyse fonctionnelle & Règles	Besoins, cas d'utilisation et contraintes métier
4	Modélisation UML	Diagramme de classes et multiplicités
5	Conception des classes	Attributs, méthodes et associations détaillées
6	Implémentation C++ & Tests	Architecture console, menu de sortie et scénarios
7	Discussion, Conclusion & Checklist	Limites, perspectives, bilan et vérifications

2. Résumé exécutif

> Ce rapport de travaux pratiques présente la démarche complète d'analyse, de modélisation et de réalisation d'une application console en langage **C++** dédiée à la gestion des activités d'une agence de voyage à Cotonou. Le système permet de centraliser la gestion des vols, des passagers, des réservations de billets et des paiements associés, tout en garantissant le respect strict des règles métier et une navigation interactive incluant une sortie contrôlée du programme.

2. Introduction et Présentation du sujet

2.1 Cadre du travail pratique

L'énoncé de ce TP impose le développement d'une application orientée objet en C++ assurant la gestion des vols et la vente de billets d'avion. La solution doit respecter les classes métier imposées, implémenter les opérations obligatoires, refléter fidèlement les multiplicités UML spécifiées et valider le fonctionnement par des scénarios de tests rigoureux.

2.2 Objectifs du TP

Objectif opérationnel	Périmètre fonctionnel
Centraliser les données	Enregistrement et mise à jour des passagers et des vols proposés.
Suivre les disponibilités	Calcul dynamique des places restantes selon les réservations actives.
Vendre les billets	Création, confirmation et annulation des réservations clients.
Tracer les paiements	Enregistrement, validation et remboursement via l'interface console C++.

2.3 Acteurs et périmètre

L'agence centralise toutes les opérations autour de trois entités principales :

Acteur	Rôle et interactions avec le système
Agent d'agence	Administre les vols, enregistre les passagers, crée ou annule les réservations, encaisse les paiements et pilote le menu principal.
Passager	Fournit ses informations d'identité, consulte les vols disponibles, recherche un trajet et effectue une réservation.
Système	Vérifie les contraintes métier (unicité des identifiants, capacité des vols), exécute les traitements et affiche les résultats.

Règle de périmètre : Les exigences explicitement formulées dans le sujet de TP priment sur les recommandations générales. Toute règle complémentaire est documentée comme hypothèse de conception.

3. Analyse fonctionnelle et Règles de gestion

3.1 Besoins fonctionnels

L'application console couvre dix fonctionnalités essentielles accessibles depuis un menu interactif en boucle :

- L'enregistrement d'un nouveau passager et la mise à jour de ses coordonnées.
- L'ajout d'un vol au catalogue, l'affichage des vols disponibles et la recherche par critère.
- La création, la consultation, l'affichage détaillé et l'annulation d'une réservation.
- L'enregistrement et le suivi d'un paiement associé à une réservation.
- La possibilité de **quitter proprement le programme** à tout moment depuis le menu principal.

3.2 Matrice des cas d'utilisation

Cas d'utilisation	Agent d'agence	Passager
Enregistrer un passager	✔	—
Ajouter un vol	✔	—
Afficher les vols disponibles	✔	✔
Rechercher un vol	✔	✔
Effectuer une réservation	✔	✔
Enregistrer un paiement	✔	—
Annuler une réservation	✔	—
Consulter les réservations	✔	✔
Afficher les informations d'une réservation	✔	✔
Quitter le programme	✔	✔

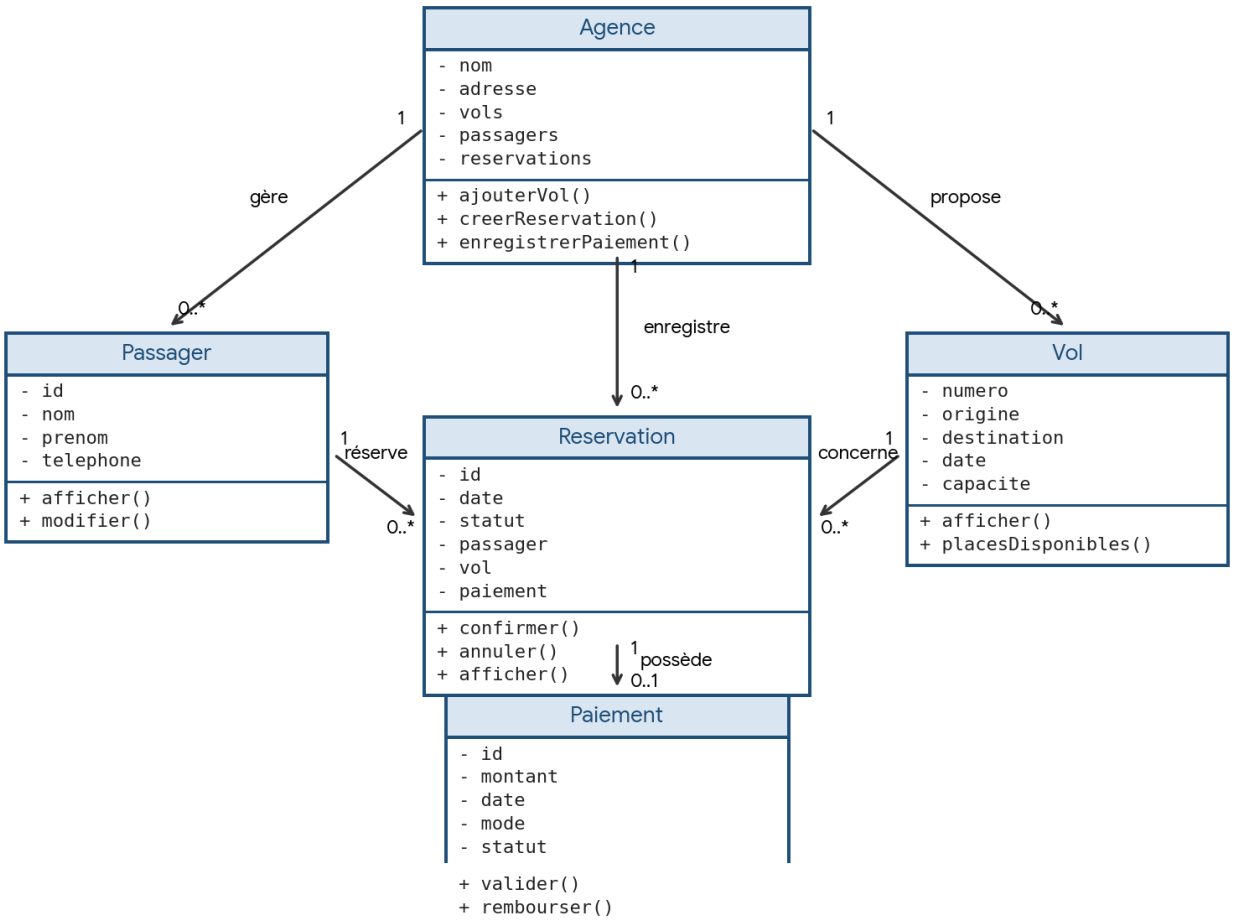
3.3 Règles de gestion

- **Intégrité des réservations :** Une réservation relie obligatoirement un passager existant à un vol enregistré.
- **Gestion des paiements :** Un paiement peut être absent (réservation en attente) ou unique par réservation (`0..1`).
- **Unicité et capacité :** Les identifiants (`id` , `numero`) doivent être uniques et le nombre de réservations confirmées ne peut jamais excéder la capacité du vol.

- **Cycle d'exécution :** Le menu principal reste actif tant que l'utilisateur ne sélectionne pas explicitement l'option de sortie.

4. Modélisation UML — Diagramme de classes

Le diagramme de classes ci-dessous formalise l'architecture orientée objet du projet, en mettant en évidence l'encapsulation des attributs privés (`-`), les méthodes publiques (`+`) ainsi que les associations et multiplicités imposées par le cahier des charges.



> **Lecture du modèle :** La classe centrale `Agence` agrège et coordonne l'ensemble des collections (`vols`, `passagers`, `reservations`). Chaque `Reservation` joue le rôle de classe pivot associant un `Passager`, un `Vol` et éventuellement un `Paiement`.

5. Conception détaillée des classes

5.1 Dictionnaire des classes métier

Classe	Attributs privés (-)	Méthodes publiques (+)
Passager	`id`, `nom`, `prenom`, `telephone`	`afficher()`, `modifier()`
Vol	`numero`, `origine`, `destination`, `date`, `capacite`	`afficher()`, `placesDisponibles()`
Reservation	`id`, `date`, `statut`, `passager`, `vol`, `paiement`	`confirmer()`, `annuler()`, `afficher()`
Paiement	`id`, `montant`, `date`, `mode`, `statut`	`valider()`, `rembourser()`
Agence	`nom`, `adresse`, `vols`, `passagers`, `reservations`	`ajouterVol()`, `creerReservation()`, `enregistrerPaiement()`

5.2 Tableau récapitulatif des associations et multiplicités

Classe source	Multiplicité	Association	Multiplicité	Classe cible
Passager	`1`	*réserve*	`0..*`	**Reservation**
Vol	`1`	*concerne*	`0..*`	**Reservation**
Reservation	`1`	*possède*	`0..1`	**Paiement**
Agence	`1`	*gère*	`0..*`	**Passager**
Agence	`1`	*propose*	`0..*`	**Vol**
Agence	`1`	*enregistre*	`0..*`	**Reservation**
Agence	`1`	*trace*	`0..*`	**Paiement**

6. Implémentation C++ et Stratégie de tests

6.1 Architecture logicielle et boucle principale

- **Découpage modulaire :** Séparation stricte entre les fichiers d'en-tête (`.h`) déclarant les interfaces des classes et les fichiers sources (`.cpp`) contenant l'implémentation métier.
- **Structures de données :** Utilisation de conteneurs standards (`std::vector`) au sein de la classe `Agence` pour stocker dynamiquement les passagers, vols et réservations.
- **Menu interactif et fermeture :** Le fichier `main.cpp` exécute une boucle `do ... while (choix != 0)` permettant d'enchaîner les opérations et proposant l'option **0. Quitter le programme** pour fermer proprement l'application et libérer la mémoire allouée.

6.2 Plan de validation et scénarios de tests

Code	Scénario testé	Action effectuée	Résultat attendu
T01	Ajout d'un vol et d'un passager	Saisie d'identifiants uniques et valides	Enregistrement confirmé
T02	Réservation nominale	Association passager existant + vol disponible	Création + places décrémentées
T03	Dépassement de capacité	Réservation sur un vol complet (`0` place)	Refus explicite avec message
T04	Paiement et confirmation	Enregistrement d'un règlement valide	Statut réservation confirmé
T05	Annulation d'une réservation	Annulation d'un billet réservé/ payé	Libération d'une place sur le vol
T06	Fermeture de l'application	Sélection de l'option *Quitter* dans le menu	Arrêt propre du programme

7. Discussion, Conclusion et Checklist finale

7.1 Discussion et limites

- **Limites actuelles :** Dans le cadre de ce TP, les données sont conservées uniquement en mémoire vive durant l'exécution du programme ; elles sont donc réinitialisées après la fermeture de l'application.
- **Perspectives d'amélioration :** L'ajout d'une persistance via des fichiers (CSV ou JSON) ou une base de données SQLite permettrait de sauvegarder l'historique de l'agence entre deux sessions.

7.2 Conclusion

> Ce travail pratique a permis de mettre en application les principes fondamentaux de la programmation orientée objet en C++ sur un cas concret de gestion d'agence de voyage à Cotonou. La phase d'analyse et de modélisation UML a assuré une structuration claire des cinq classes métier et de leurs multiplicités, facilitant l'implémentation des règles de gestion telles que le contrôle des capacités de vols et la traçabilité des paiements.
>
> L'application console réalisée répond à l'ensemble des exigences fonctionnelles du cahier des charges, offrant une interface interactive fiable et une fermeture contrôlée du programme.

7.3 Checklist finale avant remise

Critère de vérification	Statut
Page de garde conforme aux informations de l'étudiant et au cadre du TP	✔ Validé
Respect des 5 classes imposées, de leurs attributs et méthodes	✔ Validé
Diagramme de classes UML lisible avec multiplicités exactes	✔ Validé
Gestion complète des réservations, paiements et option pour quitter	✔ Validé
Scénarios de tests documentés et conclusion rédigée	✔ Validé